// DEEP DIVE

Windows Event Log Reference

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

The specific Event IDs that actually matter for detection — logons, privilege changes, process creation, persistence, and PowerShell logging — mapped directly to the attack techniques they reveal.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 — 2	Event log basics/structure, and the core logon Event IDs
3 — 5	Account/privilege, process/object, and service/persistence Event IDs
6 — 7	PowerShell logging, and mapping IDs to MITRE ATT&CK
8 — 9	A worked attack-timeline reconstruction, and a full ID reference

How to use this guide: read start to finish for the full picture, or jump straight to Section 9 for the quick-reference Event ID table. Every diagram is numbered and referenced directly in the surrounding text.

// INDEX

Table of Contents

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

// SECTION 1

Windows Event Log Basics

1.1 CHANNELS: WHERE EVENTS ACTUALLY LIVE

Windows organizes events into **channels** (sometimes still called logs). The three you'll use constantly:

CHANNEL	CONTAINS
Security	Logons, privilege use, object access — the primary channel this guide focuses on
System	OS and driver-level events, including service start/stop (Section 5)
Application	Events from installed applications
Microsoft-Windows-PowerShell/Operational	PowerShell script block and module logging (Section 6)

1.2 ANATOMY OF AN EVENT RECORD

Every event carries a consistent set of fields regardless of channel or Event ID: a **Provider** (what generated it), an **Event ID** (a numeric code identifying the specific event type), a **timestamp**, and an **Event Data** section containing fields specific to that event type — for a logon event, this includes the account name, source IP, and logon type (Section 2).

1.3 AUDITING MUST BE ENABLED

> **Not every security-relevant event is logged by default.** Windows' default audit policy is deliberately conservative — many of the Event IDs covered in this guide (especially process creation, Section 4, and PowerShell logging, Section 6) require explicit **Advanced Audit Policy** or Group Policy configuration to actually generate log entries at all. A SOC building detections around a specific Event ID must first verify the generating audit policy is actually enabled across the relevant hosts — an unlogged event is functionally invisible to a SIEM (see the SIEM & Log Analysis guide), no matter how good the correlation rule built to catch it is.

1.4 READING EVENTS AT THE COMMAND LINE

```
$ Get-WinEvent -LogName Security -MaxEvents 20
$ Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4625}
```

PowerShell's `Get-WinEvent` is the modern tool for querying event logs directly — filtering by ID, time range, or channel without needing the Event Viewer GUI, useful for both live triage and scripted collection.

// SECTION 2

Logon Event IDs

2.1 THE CORE LOGON EVENTS

EVENT ID	MEANING
4624	An account successfully logged on
4625	An account failed to log on
4634	An account logged off
4648	A logon was attempted using explicit credentials (different from the current user)
4672	Special privileges (admin-equivalent) were assigned to a new logon

2.2 LOGON TYPE: THE FIELD THAT ACTUALLY MATTERS

Both 4624 and 4625 carry a **Logon Type** field indicating *how* the logon happened — this single field is often more forensically useful than the success/failure result alone.

LOGON TYPE	MEANING
2	Interactive — physical console logon
3	Network — most SMB/file-share access, and many lateral movement techniques
4	Batch — a scheduled task
5	Service — a service starting under an account's credentials
10	RemoteInteractive — RDP

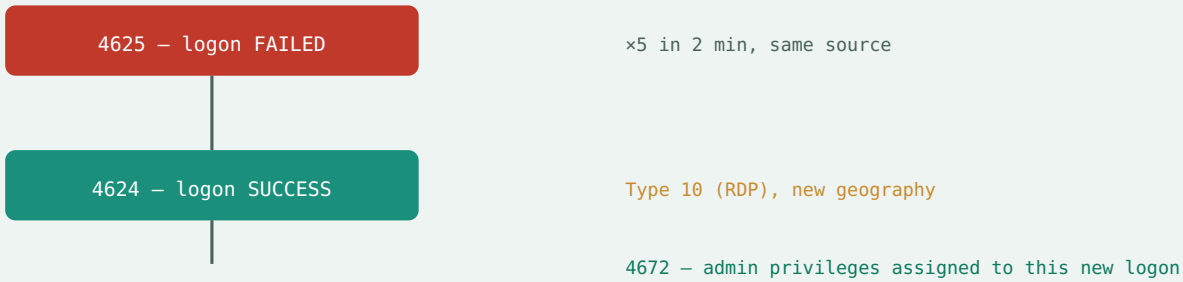

Fig 2.1 — A classic brute-force-to-compromise pattern: repeated 4625 failures, a 4624 success with a suspicious logon type, then 4672 privilege assignment.

This exact pattern is the SIEM correlation rule worked example. The failure-then-success sequence shown here is precisely the multi-event correlation rule built step by step in the SIEM & Log Analysis guide, Section 8 — this guide's Event IDs are the concrete Windows-specific version of that guide's generic "auth_failure" / "auth_success" schema.

// SECTION 3

Account & Privilege Event IDs

3.1 ACCOUNT LIFECYCLE EVENTS

EVENT ID	MEANING
4720	A user account was created
4722	A user account was enabled
4725	A user account was disabled
4726	A user account was deleted
4738	A user account was changed (password reset, attributes modified)

3.2 GROUP MEMBERSHIP EVENTS

EVENT ID	MEANING
4728	A member was added to a security-enabled global group
4732	A member was added to a security-enabled **local** group (often the local Administrators group specifically)
4756	A member was added to a security-enabled universal group

> **Event ID 4732 into the Administrators group is one of the highest-value alerts a Windows environment can have.** Legitimate admin group changes are relatively rare and usually well-documented; an unexpected 4728/4732/4756 event — especially outside a known change window, or adding an account that shouldn't need elevated access — is a strong, specific privilege escalation signal, directly analogous to the UID 0 check on Linux covered in the Linux for SOC Analysts guide, Section 4.2.

3.3 WHY BOTH ACCOUNT AND GROUP EVENTS MATTER TOGETHER

An attacker gaining a foothold often follows a predictable sequence: create a new account (4720) or compromise an existing one, then add it to a privileged group (4732) to gain administrative access without needing to actually crack or steal an existing admin's credentials directly. Watching for this specific *sequence* — not just either event type in isolation — is a much stronger detection than either alone (echoing the multi-event correlation principle from the SIEM & Log Analysis guide, Section 5.2).

// SECTION 4

Process & Object Event IDs

4.1 PROCESS CREATION: THE SINGLE MOST VALUABLE EVENT ID

Event ID 4688 (process creation) is arguably the single highest-value Windows security event for detection purposes — it records every new process launched, including its full command line (if command-line auditing is separately enabled, per Section 1.3's audit policy note), parent process, and the account that launched it.

EVENT ID	MEANING
4688	A new process was created
4689	A process exited
4663	An attempt was made to access an object (file, registry key) that has an audit rule configured

4.2 WHAT TO ACTUALLY LOOK FOR IN 4688

- **Unusual parent-child relationships** — Microsoft Office applications spawning `cmd.exe` or `powershell.exe` is a textbook exploitation/macro-malware pattern, echoing the parent-child process check from the Linux for SOC Analysts guide, Section 6.1, just on Windows.
- **Suspicious command-line arguments** — encoded PowerShell (`-EncodedCommand`), unusual download commands, or arguments designed to evade detection.
- **Processes launched from unusual locations** — an executable running from a user's Downloads folder or a temp directory, rather than a standard installation path.

4.3 EVENT ID 4663: OBJECT ACCESS AUDITING

Unlike 4688, which logs universally (once enabled), **4663** only fires for specific files, folders, or registry keys that have been explicitly configured with a **System Access Control List (SACL)** — Windows' auditing equivalent of a watch list. This makes 4663 extremely valuable for monitoring specific high-value targets (a sensitive configuration file, a credential store) without generating noise for every ordinary file access across the entire system.

> **4688 without command-line logging is far less useful.** By default, 4688 records that a process started, but not the actual arguments it was launched with — a critical piece of context for distinguishing "powershell.exe ran" (unremarkable) from "powershell.exe ran with an encoded, obfuscated command downloading from an external IP" (urgent). Enabling command-line auditing specifically (a Group Policy setting, per Section 1.3) is essential to get real value from this event.

// SECTION 5

Service & Persistence Event IDs

5.1 WHY SERVICES MATTER FOR PERSISTENCE

A Windows service, once installed, survives reboots and runs with configurable (often elevated) privileges automatically — making it a favorite persistence mechanism, directly analogous to the systemd services and cron jobs covered as Linux persistence checks in the Linux for SOC Analysts guide, Section 7.2.

EVENT ID	CHANNEL	MEANING
7045	System	A new service was installed on the system
4697	Security	A service was installed (Security-channel equivalent, requires audit policy)
7040	System	A service's start type was changed (e.g., from manual to automatic)

5.2 SCHEDULED TASKS: THE OTHER CLASSIC PERSISTENCE MECHANISM

EVENT ID	MEANING
4698	A scheduled task was created
4699	A scheduled task was deleted
4702	A scheduled task was updated

New services and scheduled tasks are both high-signal, relatively rare legitimate events. Unlike logons (Section 2), which happen constantly during normal operation, new service or scheduled task creation is comparatively infrequent in a stable, well-managed environment — making an unexpected 7045 or 4698 event a strong candidate for immediate investigation rather than something requiring heavy tuning to be actionable (see the SIEM & Log Analysis guide, Section 5.3, on rule specificity).

5.3 WHAT TO CHECK WHEN ONE FIRES

The event data for both 7045 and 4698 includes the actual binary path or command being scheduled to run — exactly the detail that separates "IT deployed a new monitoring agent" from "something installed a service pointing at an executable in a temp directory." Correlating the referenced path/command against Section 4's process creation events for that same binary often confirms whether it's legitimate software or something worth escalating immediately.

// SECTION 6

PowerShell & Script Block Logging

6.1 WHY POWERSHELL GETS ITS OWN LOGGING CHANNEL

PowerShell is both a legitimate, heavily used Windows administration tool and one of the most common vectors for "living off the land" attacks (the same concept introduced in the Linux for SOC Analysts guide, Section 6.3, just on Windows) — using entirely built-in, trusted tooling for malicious purposes rather than dropping obviously suspicious custom malware. Because of this dual nature, Microsoft built dedicated, detailed logging specifically for PowerShell activity, beyond what Event ID 4688 alone captures.

EVENT ID	CHANNEL	MEANING
4104	PowerShell/Operational	Script block logging — the actual PowerShell code that executed, including de-obfuscated content
4103	PowerShell/Operational	Module logging — which PowerShell cmdlets/pipeline execution occurred
400 / 403	Windows PowerShell	PowerShell engine state started/stopped — session boundary markers

6.2 WHY 4104 IS ESPECIALLY POWERFUL

Attackers frequently obfuscate or encode malicious PowerShell to evade simple string-matching detection (referenced in Section 4.2's "encoded PowerShell" flag). Script block logging (4104) is specifically designed to defeat this: PowerShell's own engine logs the script content *after* de-obfuscation, as it's actually about to execute — meaning even a heavily encoded or obfuscated command still produces a readable log entry showing what the code genuinely does.

> **4104 requires explicit configuration, and it can be verbose.** Like the broader audit policy point from Section 1.3, script block logging isn't on by default and must be enabled via Group Policy. Because it can generate substantial log volume in an environment with heavy legitimate PowerShell use, tuning (see the SIEM & Log Analysis guide, Section 7) matters here specifically — the goal is catching genuinely suspicious script content, not drowning in routine administrative scripting.

// SECTION 7

Mapping Event IDs to MITRE ATT&CK

7.1 WHY MAP AT ALL

The **MITRE ATT&CK framework** catalogs real-world adversary techniques in a structured, well-documented way — referenced already in the SIEM & Log Analysis guide, Section 5.3, as the recommended source for building correlation rules around actual attacker behavior rather than vague "unusual" activity. Mapping specific Event IDs to specific ATT&CK techniques turns this guide's raw ID reference into an actual detection strategy grounded in known adversary tradecraft.

7.2 DIRECT MAPPINGS

ATT&CK TECHNIQUE	RELEVANT EVENT IDS
T1078 — Valid Accounts	4624, 4625, 4648 (Section 2)
T1136 — Create Account	4720 (Section 3.1)
T1098 — Account Manipulation	4738, 4728, 4732 (Section 3)
T1059 — Command & Scripting Interpreter	4688 with command line, 4104 (Sections 4, 6)
T1543 — Create/Modify System Process	7045, 4697 (Section 5.1)
T1053 — Scheduled Task/Job	4698 (Section 5.2)
T1003 — OS Credential Dumping	4663 on LSASS-related objects (Section 4.3)

> **One technique often maps to multiple Event IDs across sections, and vice versa.** Real detection engineering rarely relies on a single Event ID for a single technique — the SIEM & Log Analysis guide's multi-event correlation principle (Section 5.2) applies directly here: a correlation rule combining 4720 *and* a subsequent 4732 (Section 3.3) is a far stronger T1098 detection than either ID alone.

7.3 USING THIS AS A STARTING POINT, NOT A CHECKLIST

ATT&CK is deliberately broad and continuously updated; this table covers the specific techniques most directly tied to the Event IDs already detailed in this guide, not the framework's full scope. The mapping exercise itself — asking "which Event IDs would actually reveal this technique?" — is a more durable skill than memorizing any specific table, since it transfers to techniques and log sources not covered here at all.

// SECTION 8

Worked Example: Reconstructing an Attack Timeline

A SIEM alert (see the SIEM & Log Analysis guide) flags unusual activity on a Windows server. Reconstructing the full timeline using Event IDs from across this guide, in the order they actually occurred:

1. **14:02 — Event ID 4625 ×6** (Section 2.1): repeated failed logon attempts against a service account, from an external IP — a brute-force attempt (T1110, closely related to T1078 from Section 7.2).
2. **14:04 — Event ID 4624** (Section 2.1), Logon Type 3 (Section 2.2): a successful network logon using that same service account — the brute-force succeeded.
3. **14:05 — Event ID 4688** (Section 4.1): `powershell.exe` launched with an encoded command-line argument, spawned directly from the logon session — T1059 (Section 7.2).
4. **14:05 — Event ID 4104** (Section 6.2): script block logging reveals the de-obfuscated PowerShell content — a download-and-execute command reaching out to an external address.
5. **14:07 — Event ID 4720** then **4732** (Section 3.1, 3.3): a new local account is created and immediately added to the local Administrators group — privilege escalation and a secondary persistence mechanism, independent of the original compromised service account.
6. **14:09 — Event ID 7045** (Section 5.1): a new service is installed, referencing a binary path in a temp directory — a third, redundant persistence mechanism.

> **Full picture:** credential brute-force → PowerShell-based initial execution → new local admin account → a backup service-based persistence mechanism, all reconstructed purely from Event IDs across six sections of this guide, in exact chronological order. This is precisely the kind of multi-stage timeline a well-tuned SIEM correlation rule (SIEM & Log Analysis guide, Section 8) is built to surface as a single, high-confidence alert rather than six separate, easy-to-miss low-priority ones.

// SECTION 9

Quick Reference & Glossary

9.1 EVERY EVENT ID IN THIS GUIDE

ID	MEANING	SECTION
4624	Successful logon	2
4625	Failed logon	2
4634	Logoff	2
4648	Logon with explicit credentials	2
4672	Special/admin privileges assigned	2
4720	Account created	3
4728 / 4732 / 4756	Member added to a privileged group	3
4738	Account changed	3
4688	Process created	4
4663	Object access (audited files/keys)	4
7045 / 4697	New service installed	5
4698	Scheduled task created	5
4104	PowerShell script block logging	6
4103	PowerShell module logging	6

9.2 GLOSSARY

TERM	DEFINITION
Channel	A category of Windows event log (e.g., Security, System).
Provider	The component that generated a given event.
Logon Type	A field indicating how a logon occurred (interactive, network, RDP, etc.).
SACL	System Access Control List — defines which object accesses trigger a 4663 audit event.
Script Block Logging	PowerShell logging that captures de-obfuscated script content as it executes.
MITRE ATT&CK	A structured, documented catalog of real-world adversary tactics and techniques.

End of guide. No single Event ID tells the whole story — as Section 8's worked timeline shows, real detections almost always come from correlating several of these together, across sections, in sequence. Knowing what each ID means is the prerequisite; knowing which combinations matter is the actual skill.